

Software Design Specification

AI-Powered API Security Testing Platform

Document Type: Software Design Specification (SDS) **Phase:** System Design **Status:** Pre-Implementation

0. Document Purpose & Scope

This document defines the system architecture for an AI-Powered API Security Testing Platform that combines traditional OWASP API Security Top 10 testing with LLM-driven, context-aware attack payload generation. It is written for a research/personal project moving from Requirement Analysis into System Design, and intentionally contains **no implementation code** — only architecture, module decomposition, data flow, diagrams, and a phased roadmap.

1. Overall System Workflow

The system workflow describes the full lifecycle of a security assessment, from API import to final report. Each stage is a discrete pipeline phase with well-defined inputs/outputs, allowing the pipeline to be paused, resumed, or partially re-run.

1.1 Stage-by-Stage Workflow

Stage 1 — API Upload The user uploads an OpenAPI/Swagger (JSON/YAML) file or a GraphQL SDL/introspection result, or provides a live endpoint URL from which the spec can be fetched. The system validates the file format, checks schema version compatibility, and stores the raw spec as an immutable artifact tied to a Project.

Stage 2 — API Parsing The appropriate parser (OpenAPI Parser or GraphQL Parser) reads the raw spec and converts it into a normalized internal representation: a list of endpoints (or GraphQL operations/types), each annotated with parameters, request/response schemas, descriptions, and security scheme references. This normalized model is what every downstream module consumes — so REST and GraphQL APIs are treated uniformly after this stage.

Stage 3 — Authentication Setup The Authentication Manager inspects declared security schemes (API Key, Bearer/JWT, OAuth2) and prompts the user to supply credentials, tokens, or a login flow. It validates the supplied credentials with a lightweight test call and stores them as an encrypted Authentication Profile linked to the project.

Stage 4 — Endpoint Extraction & Enrichment Endpoints are deduplicated, grouped by resource, and enriched with metadata: inferred sensitivity (e.g., endpoints touching user IDs, financial data), authentication requirement, and parameter types. This enriched endpoint catalog becomes the unit of work for both scanning engines.

Stage 5 — Traditional Scanning The Traditional Security Scanner runs deterministic, rule-based checks mapped to the OWASP API Top 10 (broken object-level authorization, broken authentication, excessive data exposure, rate limiting, mass assignment, security misconfiguration, injection, improper asset management, etc.). Results are stored as preliminary findings and also passed forward as *context* for the AI engine.

Stage 6 — AI Attack Generation The AI Attack Generation Engine builds a structured prompt per endpoint (or per vulnerability class) using the Prompt Builder and Context Builder, incorporating endpoint descriptions, parameter schemas, observed traditional-scan results, and prior attack/response history. The LLM Client sends this to the configured LLM and receives candidate attack payloads, which the Payload Validator checks for structural validity and safety bounds before they are queued for execution.

Stage 7 — Attack Execution The Attack Execution Engine sends both traditional and AI-generated payloads against the target API in a controlled, rate-limited, sandbox-aware manner. Every request/response pair is logged verbatim (headers, body, status, timing) as Evidence.

Stage 8 — Response Analysis The Response Analyzer inspects each response for indicators of vulnerability: anomalous status codes, data leakage patterns, error-message fingerprints, timing differentials, and schema deviations. It can optionally invoke the LLM again to reason about ambiguous responses (e.g., "does this response indicate IDOR?").

Stage 9 — Vulnerability Detection & Classification Confirmed and suspected issues are classified into discrete Findings, each mapped to an OWASP API Top 10 category, given a confidence score, and linked to the Evidence that supports it.

Stage 10 — Risk Assessment The Risk Assessment Engine scores each finding using a severity model (e.g., CVSS-inspired or a custom API-risk rubric factoring exploitability, data sensitivity, and authentication context), producing a prioritized risk list.

Stage 11 — Report Generation The Reporting Engine compiles findings, evidence, risk scores, and remediation guidance into a structured report (PDF/HTML/JSON), suitable for both technical and managerial audiences.

1.2 Workflow Properties

- **Resumability:** Each stage persists its output, so a scan can resume from any stage after failure.
- **Idempotency:** Re-running a stage with unchanged inputs produces consistent results, important for regression comparison across scans.
- **Auditability:** Every AI-generated payload and every executed request is logged for traceability and reproducibility.

2. Feature Breakdown

#	Feature	Purpose	Inputs	Outputs	Dependencies	Priority	Future Improvements	Owning Module
1	OpenAPI Import	Ingest REST API specs	OpenAPI/Swagger file or URL	Normalized endpoint model	None	P0	Support OpenAPI 3.1, \$ref resolution across files	OpenAPI Parser
2	GraphQL Import	Ingest GraphQL schemas	SDL file or introspection result	Normalized operation model	None	P0	Persisted query support	GraphQL Parser
3	Auth Detection	Identify auth scheme(s) in use	Parsed spec	List of auth requirements per endpoint	API Parsing	P0	Multi-scheme chaining	Authentication Manager
4	Credential Management	Securely store/test credentials	User-supplied tokens/keys	Encrypted Auth Profile	Auth Detection	P0	OAuth2 automated flow execution	Authentication Manager
5	Endpoint Enrichment	Classify endpoint sensitivity	Normalized endpoints	Enriched endpoint catalog	API Parsing	P1	ML-based sensitivity classifier	API Discovery Engine
6	OWASP Top 10 Scanning	Run deterministic security checks	Enriched endpoints + Auth Profile	Preliminary findings	Endpoint Enrichment	P0	Custom rule plugin system	Traditional Security Scanner
7	Context-Aware Prompt Building	Construct LLM prompts with full context	Endpoint metadata, scan results, history	Structured prompt	Traditional Scanning	P0	Adaptive prompt templates per vuln class	Prompt Builder
8	AI Payload Generation	Generate novel attack payloads	Structured prompt	Candidate payloads	Prompt Builder, LLM API	P0	Multi-model ensemble generation	Payload Generator
9	Payload Validation	Filter unsafe/invalid payloads	Candidate payloads	Validated payload set	Payload Generation	P0	Static-analysis-based safety scoring	Payload Validator
10	Attack Execution	Send payloads to target API	Validated payloads, Auth Profile	Request/response evidence	Payload Validation	P0	Adaptive rate-limiting per target	Attack Execution Engine
11	Response Analysis	Detect vulnerability indicators in responses	Evidence	Suspected findings	Attack Execution	P0	LLM-assisted ambiguous-response reasoning	Response Analyzer
12	Vulnerability Classification	Map findings to OWASP categories	Suspected findings	Classified findings	Response Analysis	P0	Confidence calibration via feedback loop	Vulnerability Detection Engine
13	Risk Scoring	Prioritize findings by severity	Classified findings	Risk-ranked findings	Vulnerability Detection	P0	Business-context-aware scoring	Risk Assessment Engine
14	Report Generation	Produce final deliverable	Risk-ranked findings + evidence	PDF/HTML/JSON report	Risk Assessment	P0	Executive summary auto-generation	Reporting Engine
15	Historical Scan Storage	Persist and compare scans over time	All scan artifacts	Scan history, diffs	Database Layer	P1	Regression/trend dashboards	Database Layer
16	Project/Workspace Management	Organize multiple APIs/scans	User actions	Project records	None	P1	Team collaboration, RBAC	User Interface / Backend

3. Major System Modules

For each module: Purpose, Responsibilities, Inputs, Outputs, Internal Components, External Dependencies, Module Communication, and Rationale.

3.1 User Interface (Frontend)

- **Purpose:** Provide the human entry point for project setup, scan configuration, monitoring, and report review.
- **Responsibilities:** Spec upload, credential entry, scan triggering, live progress display, report visualization.
- **Inputs:** User actions, backend API responses.
- **Outputs:** API requests to backend (upload, configure, start scan, fetch report).
- **Internal Components:** Project Dashboard, Spec Upload Wizard, Scan Monitor, Findings Viewer, Report Viewer.
- **External Dependencies:** React, charting library, backend REST/WebSocket API.

- **Communicates With:** Backend API Gateway only (never directly with internal modules).
- **Why it exists:** Decouples presentation from orchestration logic; allows the same backend to serve a CLI or API client later.

3.2 API Discovery Engine

- **Purpose:** Orchestrate ingestion and enrichment of API definitions.
- **Responsibilities:** Route uploads to the correct parser, deduplicate endpoints, enrich with sensitivity metadata.
- **Inputs:** Raw spec file/URL.
- **Outputs:** Normalized, enriched endpoint catalog.
- **Internal Components:** Format Detector, Endpoint Normalizer, Sensitivity Classifier.
- **External Dependencies:** OpenAPI Parser, GraphQL Parser.
- **Communicates With:** OpenAPI Parser, GraphQL Parser, Authentication Manager, Database Layer.
- **Why it exists:** Provides a single, format-agnostic entry point so downstream modules never need to know whether the source was REST or GraphQL.

3.3 OpenAPI Parser

- **Purpose:** Convert OpenAPI/Swagger specs into the internal endpoint model.
- **Responsibilities:** Resolve `$refs`, extract paths/operations/schemas/security definitions.
- **Inputs:** OpenAPI JSON/YAML.
- **Outputs:** Normalized endpoint list.
- **Internal Components:** Schema Resolver, Path Extractor, Security Scheme Extractor.
- **External Dependencies:** OpenAPI parsing library.
- **Communicates With:** API Discovery Engine.
- **Why it exists:** Isolates spec-format complexity from the rest of the system.

3.4 GraphQL Parser

- **Purpose:** Convert GraphQL SDL/introspection into the internal model.
- **Responsibilities:** Extract types, queries, mutations, subscriptions, and field-level arguments.
- **Inputs:** SDL file or introspection JSON.
- **Outputs:** Normalized operation list.
- **Internal Components:** Schema Introspector, Operation Extractor.
- **External Dependencies:** GraphQL parsing library.
- **Communicates With:** API Discovery Engine.
- **Why it exists:** GraphQL's query-based model differs structurally from REST and needs dedicated normalization logic.

3.5 Authentication Manager

- **Purpose:** Manage detection, storage, and use of authentication credentials.
- **Responsibilities:** Detect security schemes, validate credentials, refresh tokens, inject auth into outgoing requests.
- **Inputs:** Security scheme definitions, user credentials.
- **Outputs:** Authentication Profile, request-ready auth headers.
- **Internal Components:** Scheme Detector, Credential Vault, Token Refresher.
- **External Dependencies:** Encryption library, target API auth endpoints.
- **Communicates With:** API Discovery Engine, Attack Execution Engine, Traditional Security Scanner.
- **Why it exists:** Centralizing auth avoids duplicating credential-handling logic (and security risk) across multiple modules.

3.6 Traditional Security Scanner

- **Purpose:** Run deterministic OWASP API Top 10 checks.
- **Responsibilities:** Execute rule-based probes per vulnerability class; produce preliminary findings and context for the AI engine.
- **Inputs:** Enriched endpoint catalog, Auth Profile.
- **Outputs:** Preliminary findings, scan context.
- **Internal Components:** (see Section 7) sub-scanners per vulnerability category.
- **External Dependencies:** HTTP client.
- **Communicates With:** Authentication Manager, Attack Execution Engine, AI Attack Generation Engine (provides context), Database Layer.
- **Why it exists:** Deterministic checks are fast, cheap, and reliable for well-understood vulnerability classes — they shouldn't depend on LLM calls.

3.7 AI Attack Generation Engine

- **Purpose:** Generate context-aware, novel attack payloads beyond static rule coverage.
- **Responsibilities:** Build prompts, call the LLM, validate returned payloads.
- **Inputs:** Endpoint metadata, traditional-scan results, prior attack history.
- **Outputs:** Validated candidate payloads.
- **Internal Components:** (see Section 7) Prompt Builder, Context Builder, Payload Generator, Payload Validator, Prompt Templates, LLM Client.

- **External Dependencies:** OpenAI/Gemini API.
- **Communicates With:** Traditional Security Scanner (context input), Attack Execution Engine (payload output), Database Layer.
- **Why it exists:** This is the project's core novelty — moving beyond static fuzzing toward reasoning-driven, context-sensitive attack design.

3.8 Attack Execution Engine

- **Purpose:** Safely execute payloads against the target API.
- **Responsibilities:** Rate-limit requests, send payloads, capture raw evidence.
- **Inputs:** Validated payloads, Auth Profile.
- **Outputs:** Request/response evidence records.
- **Internal Components:** Request Dispatcher, Rate Limiter, Evidence Recorder.
- **External Dependencies:** HTTP client, target API.
- **Communicates With:** AI Attack Generation Engine, Traditional Security Scanner, Response Analyzer, Database Layer.
- **Why it exists:** Separating execution from generation/analysis lets execution policy (rate limits, sandboxing, retries) evolve independently of attack logic.

3.9 Response Analyzer

- **Purpose:** Interpret responses to detect vulnerability indicators.
- **Responsibilities:** Pattern-match status codes/bodies/headers, detect data leakage, optionally invoke LLM for ambiguous cases.
- **Inputs:** Evidence records.
- **Outputs:** Suspected findings.
- **Internal Components:** Pattern Matcher, Leakage Detector, Ambiguity Resolver (LLM-assisted).
- **External Dependencies:** LLM API (optional, for ambiguous cases).
- **Communicates With:** Attack Execution Engine, Vulnerability Detection Engine.
- **Why it exists:** Raw responses are not findings; this module converts evidence into interpretable signals.

3.10 Vulnerability Detection Engine

- **Purpose:** Classify and consolidate findings.
- **Responsibilities:** Map suspected findings to OWASP categories, assign confidence scores, deduplicate.
- **Inputs:** Suspected findings (from both scanner and analyzer).
- **Outputs:** Classified findings.
- **Internal Components:** Classifier, Deduplicator, Confidence Scorer.
- **External Dependencies:** None beyond internal data.
- **Communicates With:** Response Analyzer, Traditional Security Scanner, Risk Assessment Engine, Database Layer.
- **Why it exists:** Provides a single consolidation point so findings from different sources (rules vs. AI) follow one consistent taxonomy.

3.11 Risk Assessment Engine

- **Purpose:** Score and prioritize findings.
- **Responsibilities:** Apply severity rubric, factor in data sensitivity and exploitability.
- **Inputs:** Classified findings.
- **Outputs:** Risk-ranked findings.
- **Internal Components:** Severity Scorer, Prioritizer.
- **External Dependencies:** None.
- **Communicates With:** Vulnerability Detection Engine, Reporting Engine, Database Layer.
- **Why it exists:** Raw findings need prioritization for the output to be actionable rather than just a data dump.

3.12 Reporting Engine

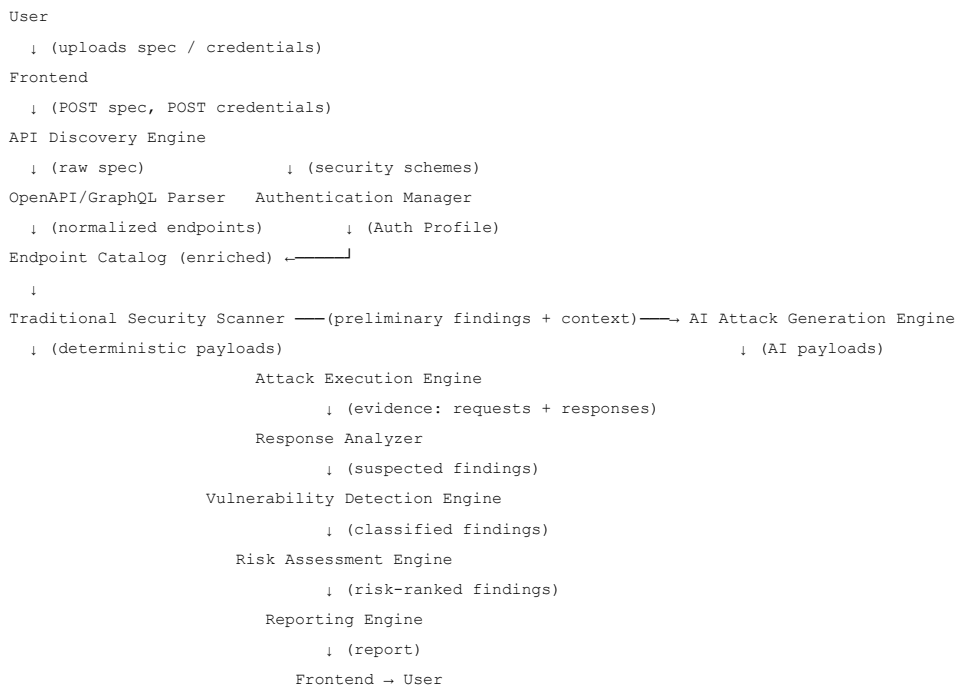
- **Purpose:** Produce the final human-readable deliverable.
- **Responsibilities:** Aggregate findings, evidence, and remediation guidance into structured documents.
- **Inputs:** Risk-ranked findings, evidence.
- **Outputs:** PDF/HTML/JSON report.
- **Internal Components:** Report Templater, Remediation Recommender, Exporter.
- **External Dependencies:** PDF/HTML rendering library.
- **Communicates With:** Risk Assessment Engine, Database Layer, User Interface.
- **Why it exists:** Separates presentation/export logic from assessment logic, enabling multiple report formats from the same data.

3.13 Database Layer

- **Purpose:** Persist all project, scan, and finding data.
- **Responsibilities:** CRUD operations, relationship integrity, historical querying.
- **Inputs:** Writes from all modules.
- **Outputs:** Reads for all modules.

- **Internal Components:** ORM models, migration scripts, query layer.
- **External Dependencies:** PostgreSQL.
- **Communicates With:** All modules.
- **Why it exists:** Single source of truth; enables historical comparison and reproducibility.

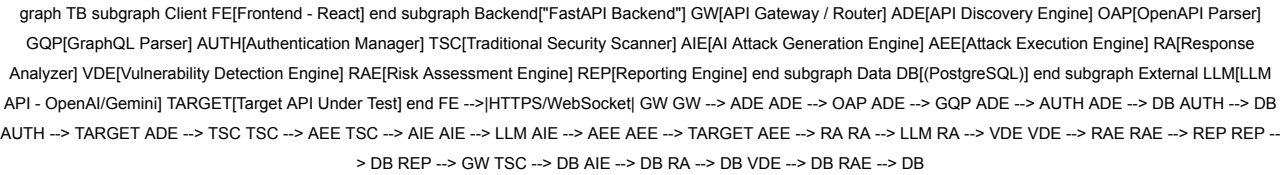
4. Data Flow Between Modules



4.1 Data Passed at Each Hop

- **Frontend → API Discovery:** raw spec file, project metadata.
- **API Discovery → Parsers:** raw spec; **Parsers → API Discovery:** normalized endpoint objects (path, method, params, schemas, descriptions).
- **API Discovery → Auth Manager:** detected security scheme definitions; **Auth Manager → API Discovery:** Auth Profile reference.
- **Endpoint Catalog → Traditional Scanner:** enriched endpoint list + Auth Profile.
- **Traditional Scanner → AI Engine:** preliminary findings, response samples, endpoint context (used as prompt context).
- **AI Engine → Attack Execution:** validated payload objects (endpoint ref, payload body, attack category, expected indicator).
- **Traditional Scanner → Attack Execution:** deterministic payload objects (same shape, different origin).
- **Attack Execution → Response Analyzer:** evidence records (request, response, timing, status).
- **Response Analyzer → Vulnerability Detection:** suspected finding objects (category guess, confidence, evidence ref).
- **Vulnerability Detection → Risk Assessment:** classified finding objects (OWASP category, confidence, affected endpoint).
- **Risk Assessment → Reporting:** risk-ranked finding objects (severity score, priority rank).
- **Reporting → Frontend:** rendered report artifact (PDF/HTML/JSON) + structured summary for UI display.
- **All modules → Database Layer:** persisted at every stage so the pipeline is resumable and auditable.

5. High-Level Architecture



Design notes: the backend is organized as independently testable modules behind a single FastAPI gateway; the LLM and target API are the only external network dependencies; PostgreSQL is the single persistence layer shared by all modules for auditability.

6. Sequence Diagram — Full Scan Lifecycle

sequenceDiagram actor User participant FE as Frontend participant BE as Backend Gateway participant AD as API Discovery participant TS as Traditional Scanner participant AI as AI Engine (LLM) participant AE as Attack Executor participant RAlz as Response Analyzer participant RA as Risk Assessment participant REP as Reporting Engine participant DB as Database participant EXT as External API User->>FE: Upload OpenAPI/GraphQL spec FE->>BE: POST /projects/{id}/spec BE->>AD: Parse & normalize spec AD->>DB: Store endpoint catalog User->>FE: Provide credentials FE->>BE: POST /projects/{id}/auth BE->>AD: Validate auth scheme AD->>EXT: Test auth call EXT-->>AD: Auth validation response AD->>DB: Store Auth Profile User->>FE: Start Scan FE->>BE: POST /projects/{id}/scan BE->>TS: Run OWASP Top 10 checks TS->>EXT: Send deterministic requests EXT-->>TS: Responses TS->>DB: Store preliminary findings BE->>AI: Generate context-aware payloads AI->>DB: Fetch endpoint + scan context AI->>AI: Build prompt (Prompt Builder) AI->>EXT: (via LLM API) request payload generation Note over AI: LLM API call shown conceptually below AI->>AE: Submit validated payloads AE->>EXT: Execute attack payloads EXT-->>AE: Responses AE->>DB: Store evidence AE->>RAlz: Forward evidence RAlz->>RAlz: Pattern/leakage analysis RAlz->>DB: Store suspected findings RAlz->>RA: Pass findings for classification & scoring RA->>DB: Store classified + risk-ranked findings RA->>REP: Trigger report generation REP->>DB: Fetch all findings + evidence REP->>FE: Return report (PDF/HTML/JSON) FE->>User: Display report

7. Detailed Module Design (Submodule Decomposition)

API Discovery Engine

- Format Detector
- Endpoint Normalizer
- Sensitivity Classifier
- Deduplication Engine

OpenAPI Parser

- Schema Resolver (`$ref` resolution)
- Path/Operation Extractor
- Security Scheme Extractor

GraphQL Parser

- Schema Introspector
- Operation Extractor (queries/mutations/subscriptions)
- Argument/Type Mapper

Authentication Manager

- Scheme Detector
- Credential Vault (encrypted storage)
- Token Refresher
- Auth Injector (attaches credentials to outgoing requests)

Traditional Security Scanner

- Broken Object-Level Authorization (BOLA/IDOR) Scanner
- Broken Authentication Scanner
- Excessive Data Exposure Scanner
- Rate Limit Tester
- Mass Assignment Scanner
- Security Misconfiguration Scanner
- Injection Scanner (SQLi, NoSQLi, command injection)
- SSRF Tester
- GraphQL-Specific Scanner (introspection exposure, query depth/complexity abuse)
- Improper Asset Management Scanner (shadow/zombie endpoint detection)

AI Attack Generation Engine

- Context Builder (assembles endpoint + prior-result context)
- Prompt Builder (combines context with templates)
- Prompt Templates (per vulnerability category)
- Payload Generator (LLM client wrapper)
- Payload Validator (structural & safety checks)
- LLM Client (provider abstraction over OpenAI/Gemini)

Attack Execution Engine

- Request Dispatcher
- Rate Limiter / Throttler
- Retry & Backoff Handler
- Evidence Recorder

Response Analyzer

- Pattern Matcher (status codes, headers, known error signatures)
- Data Leakage Detector (PII/secret pattern detection in responses)
- Timing Differential Analyzer
- Ambiguity Resolver (LLM-assisted interpretation for unclear cases)

Vulnerability Detection Engine

- Finding Classifier (maps to OWASP API Top 10)
- Deduplicator (merges findings from scanner + AI paths)
- Confidence Scorer

Risk Assessment Engine

- Severity Scorer (CVSS-inspired/custom rubric)
- Business-Context Weighter (data sensitivity, auth exposure)
- Prioritizer

Reporting Engine

- Report Templater
- Remediation Recommender
- Multi-Format Exporter (PDF/HTML/JSON)

Database Layer

- ORM Models
- Migration Manager
- Query/Repository Layer

8. Database Design

8.1 Entities

Users — Purpose: account ownership. Attributes: id, email, password_hash, created_at. Relationships: has many Projects.

Projects — Purpose: top-level container for an API assessment effort. Attributes: id, user_id, name, created_at. Relationships: belongs to User; has many API Specifications, Scans.

API Specifications — Purpose: store the raw and normalized API definition. Attributes: id, project_id, type (REST/GraphQL), raw_spec, normalized_spec, version. Relationships: belongs to Project; has many Endpoints.

Endpoints — Purpose: individual operations to be tested. Attributes: id, api_spec_id, path, method, parameters, sensitivity_level. Relationships: belongs to API Specification; has many Findings.

Authentication Profiles — Purpose: store encrypted credentials/tokens. Attributes: id, project_id, scheme_type, encrypted_credentials, status. Relationships: belongs to Project.

Payloads — Purpose: store generated/used attack payloads. Attributes: id, endpoint_id, source (traditional/AI), category, payload_body, generated_at. Relationships: belongs to Endpoint; has many Evidence records.

Scans — Purpose: represent one execution run of the pipeline. Attributes: id, project_id, status, started_at, completed_at. Relationships: belongs to Project; has many Findings, Evidence.

Findings — Purpose: confirmed/suspected vulnerabilities. Attributes: id, scan_id, endpoint_id, owasp_category, confidence_score, risk_score, description. Relationships: belongs to Scan and Endpoint; has many Evidence.

Evidence — Purpose: raw proof backing a finding. Attributes: id, finding_id, payload_id, request_raw, response_raw, timestamp. Relationships: belongs to Finding and Payload.

Reports — Purpose: generated deliverables. Attributes: id, scan_id, format, file_path/content, generated_at. Relationships: belongs to Scan.

8.2 ER Diagram

erDiagram
 USERS ||--o{ PROJECTS : owns
 PROJECTS ||--o{ API_SPECIFICATIONS : contains
 PROJECTS ||--o{ AUTHENTICATION_PROFILES : has
 PROJECTS ||--o{ SCANS : runs
 API_SPECIFICATIONS ||--o{ ENDPOINTS : defines
 ENDPOINTS ||--o{ PAYLOADS : targets
 ENDPOINTS ||--o{ FINDINGS : affected_by
 SCANS ||--o{ FINDINGS : produces
 SCANS ||--o{ EVIDENCE : generates
 SCANS ||--o{ REPORTS : results_in
 PAYLOADS ||--o{ EVIDENCE : used_in
 FINDINGS ||--o{ EVIDENCE : supported_by
 USERS {
 uuid id
 string email
 string password_hash
 timestamp created_at
 }
 PROJECTS {
 uuid id
 uuid user_id
 string name
 timestamp created_at
 }
 API_SPECIFICATIONS {
 uuid id
 uuid project_id
 string type
 text raw_spec
 json normalized_spec
 string version
 }
 ENDPOINTS {
 uuid id
 uuid api_spec_id
 string path
 string method
 json parameters
 string

```
sensitivity_level } AUTHENTICATION_PROFILES { uuid id uuid project_id string scheme_type text encrypted_credentials string status } PAYLOADS { uuid id uuid endpoint_id
string source string category text payload_body timestamp generated_at } SCANS { uuid id uuid project_id string status timestamp started_at timestamp completed_at }
FINDINGS { uuid id uuid scan_id uuid endpoint_id string owasp_category float confidence_score float risk_score text description } EVIDENCE { uuid id uuid finding_id uuid
payload_id text request_raw text response_raw timestamp timestamp } REPORTS { uuid id uuid scan_id string format text content_ref timestamp generated_at }
```

9. Design Decisions

Why FastAPI? Async-native, automatic OpenAPI generation (useful since the platform itself consumes OpenAPI specs), strong typing via Pydantic which suits a pipeline with many structured intermediate objects (endpoints, payloads, findings). Alternative considered: Flask/Django — rejected due to weaker native async support and heavier boilerplate for this I/O-bound, integration-heavy workload.

Why PostgreSQL? Strong relational integrity for the heavily-linked entity graph (Projects → Specs → Endpoints → Findings → Evidence), native JSON/JSONB support for storing flexible spec/payload structures without losing relational querying. Alternative considered: MongoDB — rejected because findings/evidence relationships benefit from foreign-key integrity and JOIN-based historical queries more than schema flexibility.

Why LLM-generated payloads (instead of purely static fuzzing)? Static payload libraries plateau against business-logic and context-dependent vulnerabilities (e.g., BOLA scenarios that depend on understanding what an ID *represents*). An LLM conditioned on endpoint semantics can propose payloads tailored to the specific resource model. Trade-off: non-determinism and potential false positives — mitigated by the Payload Validator and by keeping the Traditional Scanner as a deterministic baseline run in parallel, not replaced.

Why a modular architecture (vs. a monolithic scanner script)? Each pipeline stage (parsing, scanning, generation, execution, analysis, scoring, reporting) has independent failure modes and independent rates of change — the AI engine will iterate fastest, the parsers slowest. Modular boundaries let each evolve, be tested, and potentially be scaled independently. Trade-off: more upfront design and inter-module contracts versus a simpler single-script approach.

Why separate Attack Execution from Response Analysis? Execution is about *safely getting requests out* (rate limiting, retries, sandboxing); analysis is about *interpreting what came back*. Coupling them would force execution policy changes whenever detection logic changes, and vice versa. Separating them also allows replaying stored evidence through an improved analyzer without re-attacking the live target.

Why store raw Evidence for every request/response? Reproducibility and defensibility of findings — a security report's credibility depends on being able to show exact proof, not just a label of "vulnerable."

10. Development Roadmap

Phase 1 — API Parsing Objectives: Build OpenAPI and GraphQL parsers and the normalized internal endpoint model. Deliverables: Working parsers, normalized schema definition. Dependencies: None. Estimated Complexity: Medium. Expected Output: Ability to ingest a spec and produce a structured endpoint catalog.

Phase 2 — Endpoint Discovery & Enrichment Objectives: Deduplication, sensitivity classification, catalog enrichment. Deliverables: Enriched endpoint catalog service. Dependencies: Phase 1. Estimated Complexity: Low–Medium. Expected Output: Endpoint catalog ready for scanning.

Phase 3 — Database Layer Objectives: Implement schema from Section 8, migrations, repository layer. Deliverables: PostgreSQL schema, ORM models. Dependencies: Phase 1 (entity shapes informed by parser output). Estimated Complexity: Medium. Expected Output: Persistence available to all subsequent modules.

Phase 4 — Authentication Manager Objectives: Scheme detection, credential vault, auth injection. Deliverables: Working auth profile lifecycle. Dependencies: Phase 1, Phase 3. Estimated Complexity: Medium. Expected Output: Authenticated requests possible against target APIs.

Phase 5 — Traditional Security Scanner Objectives: Implement OWASP API Top 10 deterministic checks. Deliverables: Sub-scanners listed in Section 7. Dependencies: Phases 2–4. Estimated Complexity: High (broad surface area). Expected Output: Preliminary findings generated from rule-based scanning.

Phase 6 — AI Attack Generation Engine Objectives: Prompt Builder, Context Builder, Payload Generator, Validator, LLM Client. Deliverables: End-to-end AI payload generation pipeline. Dependencies: Phase 5 (provides context input). Estimated Complexity: High (core novelty, most iteration expected). Expected Output: Context-aware payloads validated and queued for execution.

Phase 7 — Attack Execution Engine Objectives: Dispatcher, rate limiter, evidence recorder. Deliverables: Safe, logged execution pipeline. Dependencies: Phases 5–6. Estimated Complexity: Medium. Expected Output: Evidence records stored for every executed payload.

Phase 8 — Response Analyzer, Vulnerability Detection, Risk Assessment Objectives: Build analysis → classification → scoring chain. Deliverables: Risk-ranked findings. Dependencies: Phase 7. Estimated Complexity: High. Expected Output: Prioritized, classified vulnerability list per scan.

Phase 9 — Reporting Engine Objectives: Templater, remediation recommender, multi-format exporter. Deliverables: PDF/HTML/JSON report generation. Dependencies: Phase 8. Estimated Complexity: Medium. Expected Output: Polished deliverable report.

Phase 10 — Frontend Objectives: Upload wizard, scan monitor, findings/report viewer. Deliverables: React application wired to backend API. Dependencies: All backend phases (can begin earlier against mocked endpoints). Estimated Complexity: Medium. Expected Output: Usable end-to-end product.

10.1 Cost & Tooling Considerations

This subsection documents the cost profile of the chosen stack and provides a fallback strategy for the one paid dependency in the architecture.

Free / open-source components (no cost, any usage tier): FastAPI, PostgreSQL, React, Mermaid, OpenAPI/GraphQL parsing libraries, and standard HTTP client libraries are all open-source and free to use, modify, and deploy — including for commercial or published research work.

The one paid dependency: the LLM provider The AI Attack Generation Engine is the only component with a recurring monetary cost, since OpenAI's and Google Gemini's APIs are pay-per-token (Gemini offers a usable free tier for development; OpenAI generally does not beyond a small trial credit). Because a full scan can issue many LLM calls per endpoint, free-tier quotas can be exhausted quickly even during testing.

Recommended alternative: self-hosted open-weight models (e.g., Llama) To eliminate this cost entirely during development, the LLM Client (Section 7, AI Attack Generation Engine) should be built as a **provider-agnostic interface**, allowing the underlying model to be swapped without touching the Prompt Builder, Context Builder, or Payload Validator. Concretely:

- **Local/self-hosted models** (Meta's Llama family, Mistral, etc.) can be run via a serving layer such as **Ollama**, **vLLM**, or **llama.cpp**, exposing an OpenAI-compatible HTTP API. This makes the swap nearly transparent to the rest of the pipeline — the LLM Client just points at a different base URL.
- **Cost:** zero per-token cost; the only cost is local compute (or a cloud GPU instance if your machine can't run the model size you need).
- **Trade-off:** open-weight models (especially smaller ones runnable on a laptop, e.g., Llama 3.x 8B-class) are generally weaker at structured reasoning and instruction-following than GPT-4-class or Gemini-Pro-class models, which may reduce the quality/creativity of generated attack payloads. This is a meaningful trade-off for a project whose core novelty is payload reasoning quality — worth validating with a few comparison runs before committing.
- **Suggested approach:** develop and iterate using a local Llama model to avoid burning API credits on every test run, then run final evaluation/demo scans against a hosted model (OpenAI or Gemini) to showcase best-case payload quality in your results section.

This reinforces a design decision already reflected in Section 9: the **LLM Client is an abstraction layer**, not a hard dependency on one vendor — supporting Llama, Gemini, and OpenAI behind the same interface costs nothing extra in architecture terms and removes the cost barrier during the build-out phase.

11. Cross-Cutting Concerns (Supplementary)

- **Safety/Scope Control:** the platform should only ever execute against targets the user has explicitly registered and authorized in a project (e.g., an explicit "I own/am authorized to test this target" confirmation step), with execution scoped to sandboxed or designated test environments.
- **Rate Limiting & Non-Destructiveness:** Attack Execution Engine should default to conservative request rates and avoid payloads designed to cause denial-of-service or data destruction; this should be a configurable policy, not an afterthought.
- **Observability:** structured logging across every module, correlated by scan_id, for debugging the pipeline itself.
- **Extensibility:** sub-scanners and prompt templates should be plug-in based so new OWASP categories or LLM providers can be added without touching core orchestration logic.